Projektuppgift

Frontend-baserad webbutveckling

NextTBR
Webbapplikation för böcker

Elisa Lindström

Mittuniversitetet
MID SWEDEN UNIVERSITY

Campus Härnösand Universitetsbacken 1, SE-871 88. **Campus Sundsvall** Holmgatan 10, SE-851 70 Sundsvall.
Campus Östersund Kunskapens väg 8, SE-831 25 Östersund.
Phone: +46 (0)771 97 50 00, Fax: +46 (0)771 97 50 01.

MITTUNIVERSITETET
Avdelningen för informationssystem och -teknologi

Författare: Elisa Lindström, elli1903@student.miun.se
Utbildningsprogram: Webbutveckling, 120 hp
Huvudområde: Datateknik
Termin, år: VT, 2026

Sammanfattning

Projektets syfte har varit att skapa en användarvänlig webbapplikation som visar författares kommande boksläpp. Utifrån detta togs applikationen NextTBR (Next To Be Read) fram. Applikationen kombinerar data från två API:er, Open Library API och Google Books API. Resultatet är en applikation där användare kan söka på en författare och se både dennes kommande och tidigare publicerade böcker. Sökresultatet för böcker sorteras och presenteras uppdelat i gränssnittet så att kommande boksläpp separeras från redan publicerade böcker.

Innehållsförteckning

Sammanfattning **2**

1 Inledning **4**

1.1 Problemmotivering *4*

1.2 Syfte och problemformulering *4*

2 Teori **5**

2.1 Automatiserad bygg- och publiceringsprocess *5*

2.2 API:er *5*

2.3 SASS *6*

3 Metod **7**

3.1 Förberedelser och prototyp *7*

3.2 Utveckling *7*

3.3 Data, bearbetning och lagring *8*

3.4 Testning *8*

4 Konstruktion **9**

4.1 Hämtning av data *9*

4.2 Filtrering och sortering *10*

4.3 Dynamiskt innehåll *10*

4.4 Spara böcker i localStorage *11*

4.5 Grafiska effekter *12*

5 Resultat **14**

6 Diskussion **15**

7 Litteraturförteckning **16**

8 Bilagor **18**

1 Inledning

Kulturella trender kommer och går, men de har ofta en stark påverkan på vad som intresserar oss för stunden. Böcker har funnits sedan en lång tid tillbaka men hur ”inne” det varit att läsa har varierat. Just nu syns ett ökat intresse för läsning och särskilt bland en yngre generation. Webbplatser och applikationer som Goodreads, Kindle och The StoryGraph används för att läsa, organisera och logga böcker. Sociala medier har gjort läsning mer synligt för unga där nya böcker just nu aldrig tycks sluta trenda. Samtidigt kan litteratur och kulturen runtomkring kännas seriös och svårtillgänglig.

Med ett eget bokintresse har rapporteringen kring det ökade läsintresset fångat min uppmärksamhet. Alla dessa bokapplikationer har många användbara funktioner, men ibland kan de kännas överväldigande. Ibland saknas något enklare. Utifrån detta uppkom en idé om att skapa en applikation som fokuserade främst på kommande bokpubliceringar då det var något jag själv saknat. Men det var viktigt att applikationen inte skulle kännas tråkig utan modern, enkel att förstå och rolig att använda.

I detta projekt ville jag därför ta chansen att skapa något för denna växande bokpublik. Att jag själv råkar tycka väldigt mycket om böcker blev en rolig bonus.

1.1 Problemmotivering

Ett problem med många applikationer som redan finns är att kommande boksläpp blandas med redan publicerade böcker. Detta gör att användaren själv får filtrera sökresultat vilket gör det svårare att få en överblick över kommande bokreleaser. För en yngre målgrupp som gillar att hålla sig uppdaterad ställs höga krav på att kunna tillgodogöra sig information snabbt och enkelt. Denna målgrupp befinner sig också på sociala medier och läser ofta på engelska.

NextTBR (Next To Be Read) är en applikation där användare kan söka efter en författare och direkt få upp kommande bokreleaser. Detta gör det enkelt att se vilka böcker som kommer publiceras och informationen samlas på ett ställe utan att blandas med för många andra funktionaliteter. Designen ska vara visuellt tilltalande och skapad för att främja överskådlighet.

1.2 Syfte och problemformulering

Syftet är att skapa en webbapplikation som med hjälp av kombinationen av data från olika webbtjänster kan visa användaren en specifik författares främst kommande böcker men även tidigare publicerat material. Genom att sammanställa denna information om böcker på ett och samma ställe men i tydligt separerade sektioner, beroende på publiceringsdatum, är syftet att förenkla användarupplevelsen.

Hur kan data från olika API:er kombineras för att skapa en användarvänlig webbapplikation som visar kommande bokreleaser på ett tydligt sätt?

2 Teori

2.1 Automatiserad bygg- och publiceringsprocess

Det finns ett antal verktyg och system som kan användas för att förenkla och automatisera utvecklingsmiljön vid skapandet av en applikation. För att exekvera JavaScript-kod utanför en webbläsare används verktyg som Node.js. Tillsammans med paketförvaltningsverktyget Node Package Manager (npm) kan externa bibliotek och verktyg laddas ned och användas i projektet [1]. Ett centralt sådant verktyg är Vite. Vite är ett byggverktyg som direkt gör det möjligt att koppla upp till en server under produktion och se applikationen live. Verktyget hjälper även till att optimera alla filer när applikationen är redo att byggas [2].

Genom att kontinuerligt versionshantera projektet med hjälp av Git och GitHub kan en automatiserad publiceringsprocess sättas upp. Det finns ett antal publiceringsplattformar som går att länka direkt till ett repository på GitHub. Denna koppling gör att varje gång kod ändras i ett kopplat Git-repository så kan applikationen byggas om och uppdateras utifrån den aktuella koden [3].

2.2 API:er

Ett Application Programming Interface (API) är ett bestämt regelverk för hur olika system ska kommunicera och utbyta data med varandra. Webb-API:er är en av flera olika typer av API:er men den är vanlig vid utveckling av applikationer då data kan hämtas via http, alltså över internet [4]. Utöver externa webb-API:er finns även API:er som är inbyggda i webbläsaren, så som Web Storage med sessionStorage och localStorage. Med localStorage går det att lagra data lokalt mellan sessioner, data behöver då tas bort manuellt [5].

Tidigare användes ofta tekniken AJAX för att hämta och skicka data i bakgrunden av en applikation. Idag är det mer moderna JavaScript-gränssnittet Fetch API vanligare för ändamålet. Teknikerna gör det möjligt att asynkront ladda ned data utan att webbsidan behöver laddas om [6]. För att smidigt hantera asynkrona funktioner används nyckelorden async och await. Async definierar funktionen som asynkron och await fungerar som en paus inuti i funktionen i väntan på ett resultat, ett så kallat Promise [7]. När vi vill kommunicera med ett webb-API är Fetch API användbart för att hämta data [6].

Webb-API:er kan erbjuda olika typer av dataformer för att skicka data. JavaScript Object Notation (JSON) är ett av de vanligaste moderna formaten för att utbyta data. JSON är uppbyggt utifrån JavaScript och utformat på samma sätt som ett JavaScript-objekt. Denna data kan sedan konverteras till JavaScript-objekt och implementeras i en applikation [8]. När data hela tiden hämtas och uppdateras behöver innehållet i en applikation kunna ändras. Med Document Object Model (DOM) går det att manipulera HTML dokumentets innehåll så att det uppdateras dynamiskt utifrån användarens interaktioner [9].

2.3 SASS

Sass är en CSS-preprocessor som används för att skriva CSS-kod mer effektivt. Genom funktioner som variabler, nesting och mixins går det att spara färger, hierarkiskt organisera selektorer och minska upprepningar i koden. Den nyare syntaxen som används inom Sass heter SCSS (Sassy CSS), den är också mer lik hur CSS traditionellt skrivs. SCSS måste alltid kompileras till CSS då webbläsare inte kan läsa den [10].

3 Metod

3.1 Förberedelser och prototyp

I början av projektet togs namn och en grafisk profil fram för applikationen. Eftersom målgruppen var yngre ville jag skapa något mer lekfullt med namnet. Förkortningen TBR (To Be Read) används ofta bland läscommunities på sociala medier när man pratar om böcker man ser fram emot att läsa. Förkortningen slogs ihop med ordet Next och blev tillsammans applikationens namn, NextTBR. Målgruppen som applikationen riktar sig till läser ofta på engelska, därför valdes språket engelska framför svenska för applikationen.

I detta inledande stadium användes papper och penna för att skriva ut vilka funktioner applikationen helst skulle ha. Det behövdes här göras viss research kring vilka API:er som skulle kunna fungera och vilken data de innehöll. Ganska snabbt landade beslutet på Open Library och Google Books API:er. Båda lättillgängliga och med bra guider kring parametrar som kunde användas. När jag valde bort andra API:er som New York Times Books API var det främst för att det krävdes registrerade konton och särskilda API-nycklar [11]. Open Library och Google Books API:er ställde inte samma krav och innehöll bättre data utifrån applikationens profil. Dock märkte jag längre in i projektet att Google Books API fungerade opålitligt utan en nyckel, därför skapades en API-nyckel som kunde skickas med som parameter i URL:en.

När den första prototypen togs fram i Figma var fokus på layout och färger. En av de mest centrala delarna av designarbetet är layouten, hur en webbsida ska vara uppbyggd. Valet landade på en centrerad placering av applikationens innehåll. Detta gjorde också att applikationen från början var designad för en mindre vertikal skärm. Under projektets gång gjordes vissa ändringar i prototypen (se Bilaga A för prototyper). Exempelvis var färgschemat något annorlunda inledande och kontrasten behövdes justeras i den slutgiltiga applikationen. Även extra funktioner som tillkom när webbplatsen byggdes ut, så som localStorage, fanns inte med i prototypen från början.

3.2 Utveckling

När grunden för webbplatsen var uttänkt startades ett nytt Vite-projekt upp med kommandot npm create vite@latest. Under arbetet användes en lokal server för att kunna se ändringar och testa applikationen. När applikationen var redo att publiceras genererades den optimerade versionen med kommandot npm run build. De beroenden som använts utöver Vite i projektet är JSdoc för dokumentation av JavaScript-kod, Sass för CSS-kompilering, Vite-plugin-image-optimizer för att automatiskt optimera bilder. Alla optimerade filer hamnar i mappen dist.

Applikationen är byggd utifrån en HTML-sida, index.html, som ligger i rotmappen. Då det mesta att applikationens innehåll skulle renderas med JavaScript valdes en Single Page Application (SPA) lösning framför att använda flera olika HTML-sidor [12]. Projektets syfte är också att förenkla

användarupplevelsen med en tydlig leverans av innehåll, detta motiverade även valet av SPA.

SCSS och JavaScript-filer placerades i src-mappen. Logotyp och faviconer lades i public-mappen. All JavaScript-kod lades samlad i filen main.js. SCSS-filerna är uppdelade i tre partials: base, layout och keyframes. Layout och keyframes hanterar som namnet antyder layout och animationer medan base innehåller den mer grundläggande designen. Dessa filer är sedan kopplade i filen main.scss som länkas i HTML-filen.

3.3 Data, bearbetning och lagring

Innan någon JavaScript började implementeras i projektet gjordes en grundläggande HTML-mall med tillhörande SCSS. Samtidigt behövde API:er börja testas mer grundligt. I Visual Studio Code användes Thunder Client som plugin för att se vilken typ av data som returnerades från API:erna.

Open Library visade sig ha bättre information om författare, så som födelesedatum och ibland även porträttbild. Google Books API returnerade bättre information om böcker med bland annat URL för bokomslag direkt i objektet, något som var viktigt för applikationen rent visuellt. Med Open Library behövde bokomslag och författarporträtt hämtas från ett separat API [13]. Därför valdes Open Library bort för att hämta bokdata, Google Books hade en smidigare lösning för just detta.

Data hämtades från API:erna och kombinerades i en så kallad mashup, detta gav ett mer komplett innehåll och gjorde att både författarinformation och böcker kunde visas. Särskilt Google Books API hade många parametrar som kunde hjälpa till att filtrera resultatet redan vid hämtningen [14]. Dock behövdes viss ytterligare bearbetning även när data väl hämtats då många böcker saknade omslag, publiceringsdatum och ibland titlar. För att göra applikationen mer användarvänlig användes localStorage för att göra det möjligt att spara böcker i webbläsaren.

3.4 Testning

Applikationens funktioner med sökning, sparande och borttagning av böcker testades kontinuerligt under projektet och mot slutet även av en testperson. I slutskedet gjordes webbläsartester i Chrome, Firefox, Safari och Edge. Applikationen testades även på enheterna iPhone, Android, Mac och PC. Särskilt testerna på iPhone var givande då ändringar gällande sökrutan kunde göras för att förhindra Safari från att zooma in för mycket när fältet var aktivt.

4 Konstruktion

I projektet har en webbapplikation som hämtar data från två olika API:er utvecklats. När användaren söker efter en författare på webbsidan hämtas först data om författaren från ett API och sedan används författarens namn för att hämta böcker från ett annat API. Applikationens innehåll genereras huvudsakligen dynamiskt med hjälp av JavaScript. Utöver detta har även grafiska effekter och animationer implementerats genomgående i applikationen.

4.1 Hämtning av data

När användaren besöker webbplatsen syns en rubrik, kort information om vad applikationen gör samt sökfält. Sökfältet är kopplat till ett formulär som vid antingen enter eller klick på sökknappen startar i gång API-flödet. Användaren fyller i önskad författare och värdet skickas därefter till funktionen fetchAuthor (se Figur 1). Med ett Fetch-anrop hämtas data från Open Library API och returneras som ett JSON-objekt innehållandes information om författaren. Informationen som används är författarens namn, ID, födelsedatum samt mest kända verk.

```
async function fetchAuthor(authorValue) {
  try {
    const response = await
fetch(`https://openlibrary.org/search/authors.json?q=${authorValue}`);
    const author = await response.json();

    if (author.docs.length === 0) {
      displayAuthorNotFound();
      return;
    }
    displayAuthor(author.docs[0]);
  } catch (error) {
    console.error("Something went wrong:", error);
  }
}
```

Figur 1. Asynkront Fetch API med try/catch och aync/await.

När författaren har hittats skickas data vidare till en funktion som skriver ut författarinformationen genom att HTML-element skapas. Även en bild på författaren hämtas med hjälp av ett unikt författar-ID. Bilden hämtas från Open Librarys separata API för bilder och om ingen bild skulle hittas hanteras detta med en OnErrorEventHandler. Om användarens inputvärde (författaren) inte skulle gå att hitta hos Open Library hanteras detta med hjälp av en separat

funktion som skriver ut att ingen författare kunnat hittas och applikationen stoppas.

Om författare kunnat hittas görs sedan ett nytt anrop, denna gång till Google Books API. Från Google Books hämtas böcker som innehåller författarens namn. För att hitta rätt böcker skickas ett antal parametrar med i anropet. Parametern inauthor användes för att precisera att endast böcker som i fältet ”author” innehåller författarens namn skulle tas med. Detta filtrerade bort vissa böcker som skulle kunna ha författarens namn endast i titeln. Parametern printType hjälpte till att endast ta med ”books” och filtrera bort exempelvis tidningar. Parametern maxResults styrde hur många resultat anropet skulle returnera, här valdes 20.

4.2 Filtrering och sortering

Utöver den filtrering som gjordes direkt i fetch-anropet behövde böckerna som returnerades från Google Books API viss ytterligare filtrering. Därför skapades en funktion som tar emot en array av bokobjekt och filtrerar dessa utifrån om boken har titel, publiceringsdatum och omslag. Utöver titel var datum och omslag särskilt viktigt. Datum behövdes för att i nästa steg kunna sortera böckerna utifrån om bokens publiceringsdatum låg i dåtid eller framtid.

Efter filtreringen skickades filtrerad data vidare till en funktion för sortering. JavaScripts inbyggda funktion new Date() användes för att skapa ett datumobjekt med nuvarande datum och tid. Datumobjektet användes sedan för att hela tiden jämföra böckernas publiceringsdatum med det nuvarande datumet och på så vis dela upp böcker i två separata arrayer. Redan publicerade böcker sorterades även så de skulle visas med senast publicerad först. Detta för att listan över publicerade böcker kunde bli lång medan kommande releaser oftast bara innehåller max en eller två böcker.

4.3 Dynamiskt innehåll

Efter filtrering och sortering skickas arrayerna vidare till två separata funktioner för rendering, beroende på om böckerna är redan publicerade eller kommande. Funktionerna displayUpcomingBooks och displayPublishedBooks är uppbyggda på liknande sätt men de är kopplade till olika sektioner på webbsidan. Sektionerna har hittills varit dolda med hjälp av CSS-klassen ”hidden”. När böcker har hämtats och skickats till respektive funktion tas denna klass bort med hjälp av classList.remove, vilket gör att innehållet nu syns. Det skapas sedan och skrivs ut sektionsrubriker.

I slutet av funktionerna används en forEach-loop som för varje book i arrayen anropar en separat funktion som skapar själva bokkortet. Funktionerna hanterar även tomma arrayer genom en return. Funktionen som skapar bokkort ansvarar för att skapa HTML-element med textContent baserat på utvald information som finns i varje bokobjekt. De delar som används är: bokomslag, titel, författarens namn, publiceringsdatum, antal sidor samt en länk till boken på Google Books.

För varje bok skapas ett div-element med variabelnamnet bookCard. Det är innanför detta element som all information om en bok läggs till. Vid hämtning av omslagsbild gjordes detta automatiskt via HTTP-anslutning vilket konsolen varnade för. Därför lades en replace in i omslagets URL som ändrade http till HTTPS. Ibland saknades delar av data som inte kontrollerats i tidigare filterfunktion, detta hanteras då med en if-sats (se exempel i Figur 2).

```
if (pages && pages > 0) {
    pageCountEl.textContent = `${pages} pages`;
    bookInfoEl.appendChild(pageCountEl);
}
```

Figur 2. Kontroll av data gällande antal sidor i bok

4.4 Spara böcker i localStorage

Varje bokkort innehåller en knapp som gör det möjligt att spara böcker i localStorage. Även om sidan laddas om eller användaren kommer tillbaka till applikationen vid ett senare tillfälle finns sparade böcker då kvar. När användaren klickar på knappen ”Want to Read” anropas funktionen saveBookCard. Denna funktion skapar ett nytt objekt (se Figur 3) som innehåller bokens titel, författare, publiceringsdatum och en länk till boken på Google Books. Innan boken objektet lagrad görs en kontroll för att se om någon bok med samma titel redan finns i localStorage. Det går inte att lagra en bok med samma titel flera gånger. Om boken inte redan är lagrad kommer den att sparas med unshift, vilket gör att boken hamnar först om det redan finns andra lagrade böcker i arrayen. Knappen i bokkortet ändrar text till ”Saved” och blir inaktiverad.

```
const bookData = {
    title: book.volumeInfo.title,
    author: book.volumeInfo.authors,
    date: book.volumeInfo.publishedDate,
    link: book.volumeInfo.infoLink
};
```

Figur 3. Modell för lagrat bokobjekt

För att visa sparade böcker används funktionen displaySavedBooks. Funktionen hämtar data från localStorage och är kopplad till en sektion där sparade böcker ska visas. Om inga sparade böcker finns kommer sektionen vara dold. När en bok blir lagrad i localStorage kommer sektionen att bli synlig följt av en rubrik med texten ”Want to Read”. Under rubriken visas sedan den eller de böcker som sparats.

För varje sparad bok kommer funktionen att anropa en separat funktion createSavedBookCard, som skapar själva bokkortet för den sparade boken. Detta kort skiljer sig från det kort som skapas för sökresultatets böcker, det är mindre

och innehåller färre detaljer om den sparade boken. Dessutom hämtas data inte från ett API i detta fall utan från localStorage.

Varje bokkort har en knapp med texten ”Delete”, denna knapp tar bort bokobjektet från localStorage. När användaren trycker på knappen uppdateras innehållet dynamiskt genom att först anropa funktionen displaySavedBooks som kontrollerar vilka böcker som finns sparade. Sedan anropa funktionen sortBooks och skicka med variabeln currentBooks. Variabeln currentBooks används som en lagring av den senast hämtade listan med böcker från Google Books API. Detta gör att webbsidans sökresultat kan uppdateras utan att ett nytt fetch-anrop behöver göras. I stället uppdateras det senaste sökresultatet direkt och knappar som var ”Saved” ändras tillbaka till ”Want to Read” om användaren tagit bort en bok från listan utan att göra en ny sökning.

4.5 Grafiska effekter

För att göra applikationen mer levande användes SCSS för styling och animationer. Målet var att skapa en minimalistisk design som hade inslag av rörelse och färg. Efter mycket testande föll valet av typsnitt på Inter Tight från Google Fonts som sparades i SASS-variabeln $font-stack. Detta typsnitt användes genomgående både på rubriker och brödtext. Eftersom applikationen ska fungera på olika skärmstorlekar behövde typsnittet vara enkelt att läsa både på mobil och större skärm.

Bakgrundsfärg och sekundärfärg sparades i SASS-variabler för att kunna återanvändas i olika delar av webbplatsen. För återkommande komponenter så som kort skapades en @mixin som gick att återanvända för bokkort, författarprofil och sparade böcker. Även flex-center skapades som en @mixin då en centrerad flexbox användes genomgående för både header, footer och huvudinnehåll.

Flera animeringar och visuella effekter har använts för att göra applikationen mer interaktiv. Den första animationen som syns är att allt innehåll på sidan ligger innanför en animation-wrapper med fade in-effekt som går från 0 till 1 i opacitet på 1.5 sekunder. Vilket gör att innehållet mjukt laddas på sidan. Sökknappens färg justeras med hjälp av color.scale där grundfärgens ljushet justeras för att skapa en gradient. Alla knappar har en viss förstoring vid hover som gör att de mjukt skalas 1.05 när användaren för musen över knappen. Bokkorten har en hoverfunktion som gör att de förflyttas -4px uppåt så länge muspekaren är ovanpå kortet.

Anropen till både Open Library och Google Books kunde ibland ta lite tid, därför lades en rund loader till i väntan på sökresultat. För loadern skapades en @keyframes som går från 0deg till 360deg (se Figur 4) och används infinite, alltså skapar en konstant rotation. Loadern är inkorporerad i JavaScript-koden där den visas direkt när en sökning görs och döljs när böcker hämtats, alltså fram tills att allt resultat är klart. Den visar användaren att något händer på sidan även när innehållet håller på att laddas.

```
@keyframes rotate {
```

```
    from {
        transform: rotate(0deg);
    }

    to {
        transform: rotate(360deg);
    }
}
```

Figur 4. Keyframes för loader

5 Resultat

Webbapplikationen har en sökfunktion som utifrån en författares för och efternamn gör det möjligt att söka efter både kommande och redan publicerade böcker. När användaren sökt på en författare kommer först en författarprofil fram som hämtar data från Open Library API. Sedan visas sökresultatet för böcker med data hämtad från Google Books API uppdelat i två sektioner. Om inga kommande böcker kan hittas visas detta genom rubriken ”No Upcoming Releases Found”. Denna uppdelade design var viktig utifrån projektets syfte att förenkla användarupplevelsen just kring hur kommande boksläpp kommuniceras och inte blanda alla sökresultat i samma sektion. Mashupen visar även hur kombinerad data bidrog till ett mer heltäckande innehåll i applikationen där både information om författare och böcker kunde hämtas.

Utöver sökfunktionen går det även att spara böcker i en lista. Denna lista sparas mellan sessioner och användaren kan komma tillbaka till applikationen vid ett senare tillfälle och listan finns då sparad. Med en funktion som detta kunde just den användarvänliga delen av projektets syfte förstärkas ytterligare. Genom att kunna söka information, se den informationen, men också spara delar av informationen blir upplevelsen mer interaktiv. Även detaljer som loader, animerade rörelser och hur innehållet laddas och uppdateras dynamiskt bidrog till just användarvänlighet.

6 Diskussion

En stor del av arbetet har gått åt till att systematiskt arbeta fram en fungerande kommunikation mellan funktioner i JavaScript. När små detaljer ändras i en funktion kan detta leda till krav på större förändringar i en annan funktion. Precis som i verkligheten är kommunikationen mellan funktioner inte linjär, funktioner kräver samspel och kommunicerar ofta fram och tillbaka med varandra. Funktioner kräver ömsesidighet och med en bakgrund inom kommunikationsvetenskap har detta tankesätt till viss del hjälpt mig förstå vad som händer bättre.

Bland det viktigaste i ett sådant här projekt är planeringen innan man börjar koda. Just för applikationer, där flödet mellan funktioner är bland det mest centrala, blir det viktigare än någonsin. Även om planering gjordes både av funktioner och design kommer jag ta med mig att bättre planering av JavaScript-kod hade kunnat underlätta arbetet ytterligare.

De flesta webb-API:er går att styra med hjälp av parametrar, filtrering och sortering, dock är det svårt att hitta en helt perfekt webbtjänst. I dessa fall är en mashup en bra lösning, men även då kanske vissa saker behöver anpassas efter API:erna snarare än tvärtom. Där underlättade projektidéns flexibilitet och att applikationen kunde anpassas under arbetets gång utifrån den data som gick att hämta.

Det roligaste med att bygga en sådan här typ av applikation är att det finns stora möjligheter för fortsatt utveckling. Jag har flera idéer som exempelvis en separat sida för sparade böcker, där bokreleaser sorteras efter månad vilket enkelt skapar en bättre överblick. Det skulle också gå att utveckla författarsektionen, länka till författarens sociala medier eller webbplats, hämta senaste nyheter kopplade till författaren, bygga in filtrering direkt i sökformuläret så att användaren själv kan styra sökresultatet. Det finns mycket man kan göra med denna typ av applikation. Endast fantasin sätter gränser (och tidsramen…). Ibland kan också en enklare typ av applikation vara syftet vilket var lite grundtanken med NextTBR.

7 Litteraturförteckning

[1] Canvas, Frontend-baserad webbutveckling. Modul: 1, ”NodeJs och NPM,” 2026. [Online]. Available: https://canvas.miun.se/courses/8221/pages/nodejs-och-npm?module_item_id=195258. [Använd 26 Mars 2026].

[2] Canvas, Frontend-baserad webbutveckling. Modul: 1, ”Automatisering med Vite,” 2026. [Online]. Available: https://canvas.miun.se/courses/8221/pages/automatisering-med-vite?module_item_id=195261. [Använd 26 Mars 2026].

[3] Canvas, Frontend-baserad webbutveckling. Moment: 1, ”Automatiserad publicering & publiceringsplattformar,” 2026. [Online]. Available: https://canvas.miun.se/courses/8221/pages/automatiserad-publicering-and-publiceringsplattformar?module_item_id=195263. [Använd 26 Mars 2026].

[4] Canvas, Introduktion till programmering i JavaScript. Moment: 5.1, ”Webbtjänst och API,” 2025. [Online]. Available: https://canvas.miun.se/courses/6149/pages/webbtjanst-och-api?module_item_id=141029. [Använd 26 Mars 2026].

[5] MDN, ”Using the Web Storage API,” 22 Februari 2025. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API/Using_the_Web_Storage_API. [Använd 26 Mars 2026].

[6] Geeks for Geeks, ”Difference Between AJAX And Fetch API,” 5 Augusti 2025. [Online]. Available: https://www.geeksforgeeks.org/javascript/difference-between-ajax-and-fetch-api/. [Använd 26 Mars 2026].

[7] Canvas, Frontend-baserad webbutveckling. Moment: 2, ”Async / Await,” 2026. [Online]. Available: https://canvas.miun.se/courses/8221/pages/async-slash-await?module_item_id=198900. [Använd 28 Mars 2026].

[8] Canvas, Introduktion till programmering i JavaScript. Moment: 5.1, ”JSON (JavaScript Object Notation),” 2025. [Online]. Available: https://canvas.miun.se/courses/6149/pages/json-javascript-object-notation?module_item_id=141027. [Använd 26 Mars 2026].

[9] Geeks for Geeks, ”HTML DOM (Document Object Model),” 5 Februari 2026. [Online]. Available: https://www.geeksforgeeks.org/javascript/dom-document-object-model/. [Använd 26 Mars 2026].

[10] Canvas, Frontend-baserad webbutveckling. Moment: 3, ”CSS-preprocessorer,” 2026. [Online]. Available: https://canvas.miun.se/courses/8221/pages/css-preprocessorer?module_item_id=199995. [Använd 26 Mars 2026].

[11] The New York Times Developer Network, ”Books API,” 2026. [Online]. Available: https://developer.nytimes.com/docs/books-product/1/overview. [Använd 27 Mars 2026].

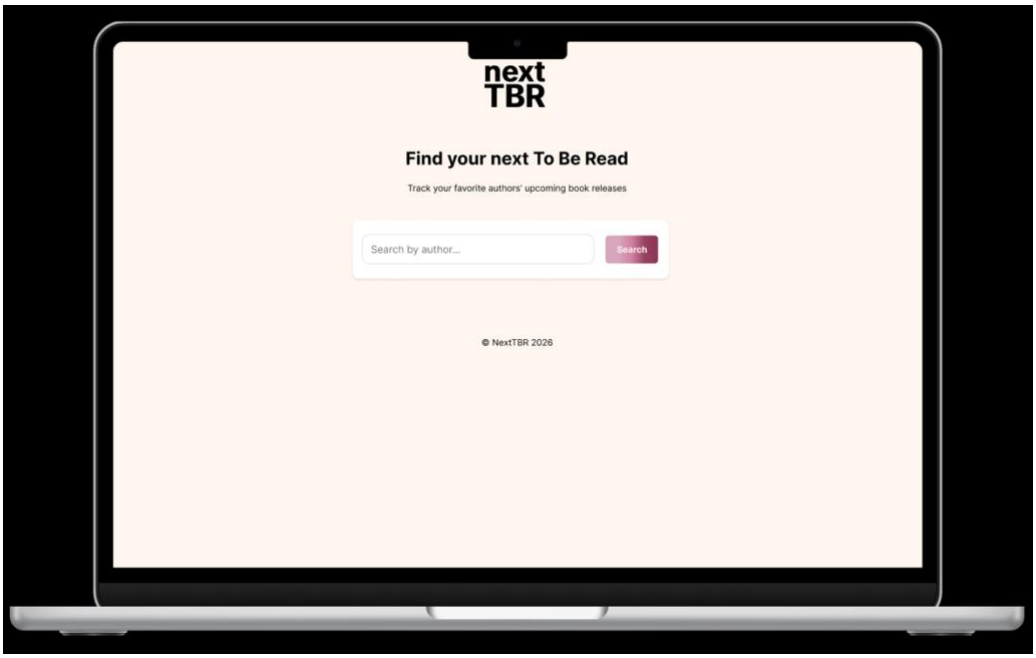
[12] Geeks for Geeks, ”SPA vs MPA: Which One is Better For You?,” 5 Augusti 2025. [Online]. Available: https://www.geeksforgeeks.org/blogs/spa-vs-mpa-which-one-is-better-for-you/. [Använd Mars 27 2026].

[13] Open Library, ”Open Library Covers API,” 13 Augusti 2024. [Online]. Available: https://openlibrary.org/dev/docs/api/covers. [Använd 26 Mars 2026].

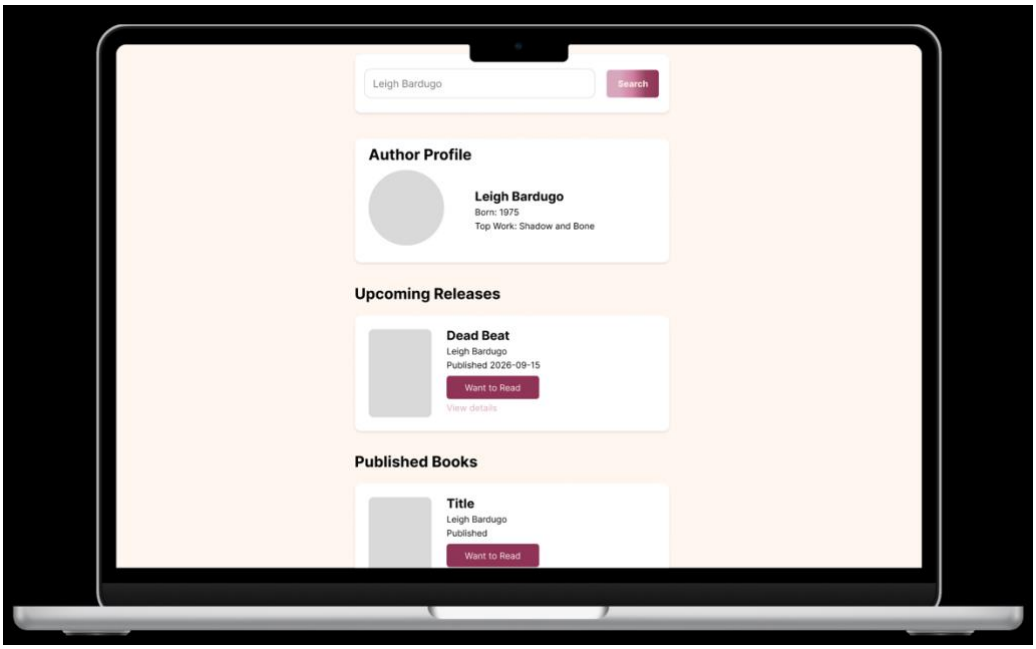
[14] Google Developers, ”Using the API,” 28 Augusti 2025. [Online]. Available: https://developers.google.com/books/docs/v1/using. [Använd 26 Mars 2026].

8 Bilagor

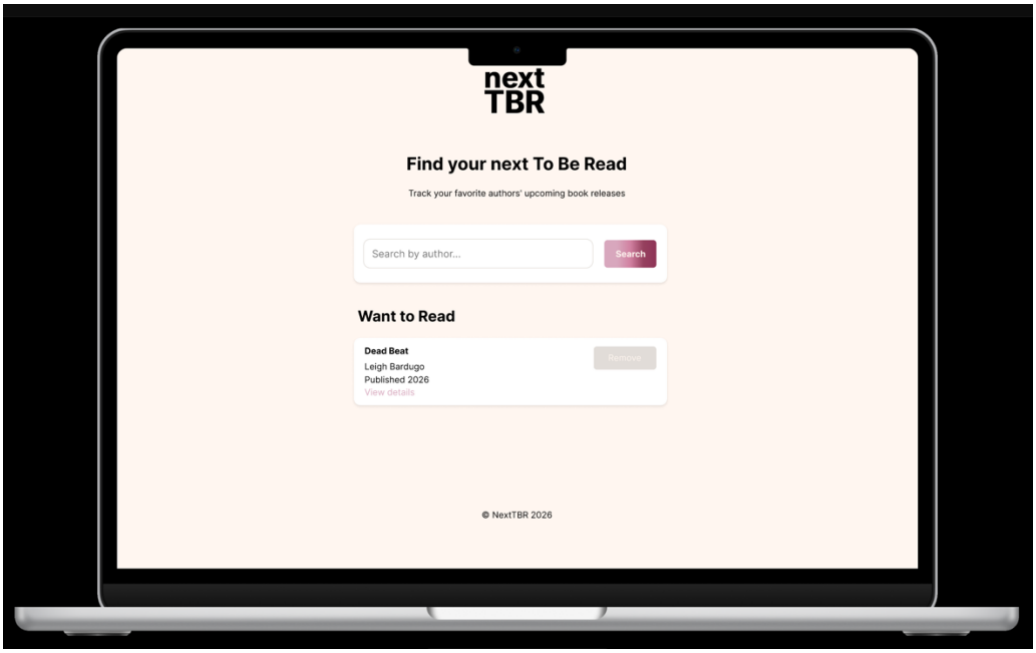
Bilaga A. Figma-prototyper desktop



Startsida



Sökresultat


next
TBR
Find your next To Be Read
Track your favorite authors' upcoming book releases
Search by author...
Search
Want to Read
Dead Beat
Leigh Bardugo
Published 2026
View details
Remove
© NextTBR 2026

Sparad bok